

Michael Kelley
UCLA Agentic AI

Reflection: Building a Multi-Agent Review System

The core design decision was to decompose the literature review task into four specialized agents rather than assigning the entire workflow to a single agent. This mirrors how a research team actually operates. Having a strategist, a researcher, an analyst, and a writer allows each expert to focus on their own portion of the work where they excel the most.

A single agent handling the full pipeline would run into compounding problems as they're too limited just how much quality work they can produce. By delegating each responsibility to a dedicated agent, I was able to define the quality bar for each stage of the process independently and catch failures at handoff points instead of at the end.

The Query Expander was designed with an obsession for source diversity, its goal explicitly forbids overlapping keywords across sub-questions because a literature review that draws from one or two papers repeatedly is a failure of research. The Source Hunter was given a strict rule against using the same paper twice, grounded in the backstory of being a master in finding exact passages across a wide body of academic text. The Synthesizer was constrained to only work with what the Source Hunter returned, preventing it from using training data instead of the corpus. The Report Writer was held to the same constraint, with an instruction to write "source unclear" rather than invent a citation when evidence was absent.

There were several significant design decisions that involved real tradeoffs between approaches. First, sequential v. hierarchical process. CrewAI supports both processes. A hierarchical process uses a manager agent to delegate the tasks dynamically and can re-run any agent if an output is unsatisfactory. I chose sequential because it is more predictable and easier to debug. Each agent runs exactly once in a defined order and the tradeoff is that there is no feedback loop. If the Source Hunter had returned poor results, the Synthesizer and Report Writer couldn't send it back. A hierarchical process handles this more gracefully but is unpredictable and costs more tokens.

Secondly, memory on v. memory off. This was the most extensively tested tradeoff in the project. Memory on allowed agents to build on previous runs, improving citation consistency on repeated requests. However, it also caused contamination. Bad retrievals from earlier runs were stored and reused in later runs, which only reinforced errors. Memory off produced cleaner first runs but removed the benefit of accumulated context. The final decision was to keep memory on with periodic manual clearing between questions when it started to get contaminated.

Third, generalist v. specialist agent descriptions. Early versions of the agents had broad, generic goals which produced subpar outputs because the LLM had too much latitude to interpret the task. Switching to highly specified goals improved the consistency and output dramatically. The tradeoff is if the question doesn't fit the assumed structure, then the agent struggles with a good output. A more adaptive system is most ideal.

The Synthesizer agent provided a clear example of ReAct-style reasoning. After receiving the retrieved passages, it did not simply summarize them. Instead, it reasoned about what it needed, specific paper-to-paper tension. It acted by calling `search_memory` with target queries including "disagreements between Wooldridge and Xi" and "Wang vs RAG Survey

performance”. It then observed the returned memories which confirmed tension between the papers and used the observation to construct the themes.

The sequential pipeline worked well. Context passed between tasks, and the four-agent structure produced quality outputs after testing and refining. The biggest change was the synthesizer task changes. I required paper-to-paper tensions and unanswered questions for every theme. It pushed the output from summarization to genuine critical analysis.

The most persistent problem was citation accuracy. I had originally structured the citations to be (Author, year), but author names were constantly wrong or missing and some themes contained uncited claims despite specific instructions. After extensive testing, I believe the root cause is the RAG tool’s output returns paper titles but not author metadata. Agents receiving search results had no reliable author information so they either guessed or left fields blank. So I had to manually input the papers from the corpus in the `expand_task` description and change how the citations were created.

A related issue was memory contamination. When memory was enabled across multiple runs on the same question, previous low quality retrievals were stored and reused. Clearing the memory between re-runs helped, but highlighted that memory needs careful management. Prompt precision was also very demanding. Early agent descriptions produced generic outputs because the goals were too broad. It took multiple iterations with increasingly specific language to get the agents to behave consistently.

Deploying the system in production would require several changes. The most important is fixing the author metadata as well as memory management. A citation verification agent would be of value for correctly identifying when the authors are being cited properly or not. It could flag when they don’t correctly match and would help with the hallucination problem. Memory management would need a more sophisticated strategy. A production system should manage namespace memories by topic and flag low confidence retrievals to prevent contamination. Another production consideration is making the system more dynamic. The current fixed prompt structure works well for broad survey questions but struggles with specific or comparative ones. A routing agent that classifies the incoming question and adjusts prompts accordingly would make the system robust across question types rather than optimized for one style.

Overall, this project demonstrated both the power and complexity of multi-agent systems. The delegation model produced noticeably better literature reviews than a single agent could, and the iterative refinement of the agent roles showed how much output quality depends on the precision. The challenges encountered are not unique to this implementation. They reflect limitations of current LLM-based systems that any production deployment would need to address. Working through them helped provide a better understanding of where these systems succeed, where they fail, and what work remains before they can be trusted. This assignment made clear that building reliable multi-agent systems is as much about careful prompt engineering and infrastructure design as it is about the agents themselves.